

控制语句和逻辑思维

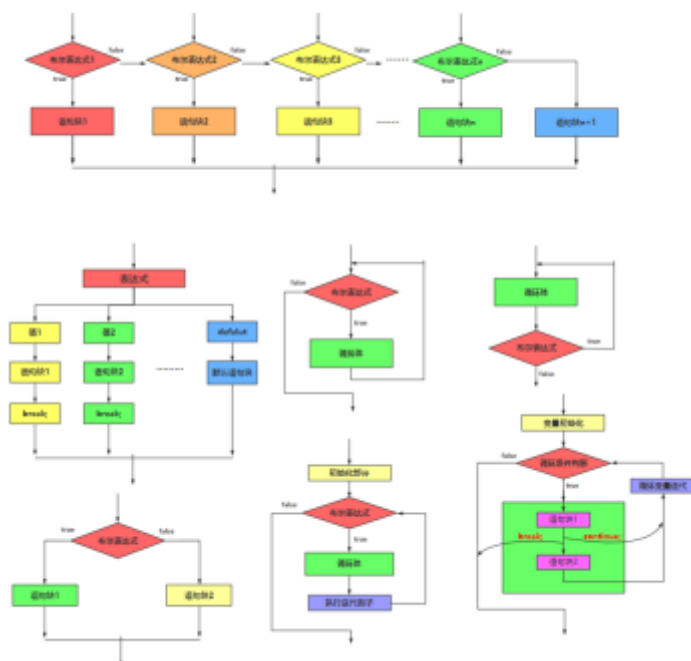


控制语句：把语句组合成能完成一定功能的小逻辑模块。

分为三类：顺序、选择和循环。

学会控制语句，是真正跨入编程界的“门槛”，是成为“程序猿”的“门票”。

- ① “顺序结构”代表 “先执行a，再执行b” 的逻辑。比如，先找个女朋友，再给女朋友打电话；先订婚，再结婚；
- ② “条件判断结构”代表 “如果...，则...” 的逻辑。比如，如果女朋友来电，则迅速接电话；如果看到红灯，则停车；
- ③ “循环结构”代表 “如果...，则重复执行...” 的逻辑。比如，如果没打通女朋友电话，则再继续打一次；如果没找到喜欢的人，则再继续找



很神奇的是，三种流程控制语句就能表示所有的事情！

不信，你可以试试拆分你遇到的各种事情。实际上，任何软件和程序，小到一个练习，大到一个操作系统，本质上都是由“变量、选择语句、循环语句”组成。



实时效果反馈

1. 条件判断语句，代表现实中的什么逻辑？

- A "先执行a，再执行b"
- B "如果...，则..."
- C "如果...，则重复执行..."
- D 必须执行

2. 循环语句，代表现实中的什么逻辑？

- A "先执行a，再执行b"
- B "如果...，则..."
- C "如果...，则重复执行..."
- D 必须执行

答案

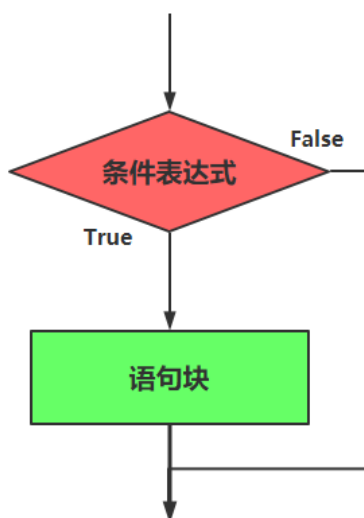
1=>B 2=>C

选择结构(条件判断结构)



选择结构通过判断条件是否成立，来决定执行哪个分支。选择结构有多种形式，分为：单分支、双分支、多分支。

单分支选择结构



if语句单分支结构的语法形式如下：

```
1 if 条件表达式:
2     语句/语句块
```

- ① 条件表达式：可以是逻辑表达式、关系表达式、算术表达式等等。
- ② 语句/语句块：可以是一条语句，也可以是多条语句。多条语句，缩进必须对齐一致

【操作】输入一个数字，小于10，则打印这个数字(if_test01.py)

```
1 num = input("输入一个数字： ")
2 if int(num)<10:
3     print("小于10的数： "+num)
```

条件表达式详解

在选择和循环结构中，条件表达式的值为 `False` 的情况如下：

False、0、0.0、空值None、空序列对象（空列表、空元祖、空集合、空字典、空字符串）、空range对象、空迭代对象。

其他情况，均为 `True`。这么看来，Python所有的合法表达式都可以看做条件表达式，甚至包括函数调用的表达式。

【操作】测试各种条件表达式

```
1 if 3:      #整数作为条件表达式
2     print("ok")
3 a = []     #列表作为条件表达式，由于为空列表，是
4             False
5 if a:
6     print("空列表，False")
7 s = "False" #非空字符串，是True
8 if s:
9     print("非空字符串，是True")
```

```
9
10 c = 9
11 if 3<c<20:
12     print("3<c<20")
13 if 3<c and c<20:
14     print("3<c and c<20")
15
16 if True:          #布尔值
17     print("True")
```

⚠ 条件表达式中，不能有赋值操作符 =

在Python中，条件表达式不能出现赋值操作符 =，避免了其他语言中经常误将关系运算符 == 写作赋值运算符 = 带来的困扰。如下代码将会报语法错误：

```
1 if 3 < c and (c=20):    #直接报语法错误！
2     print("赋值符不能出现在条件表达式中")
```

实时效果反馈

1. 如下代码，打印输出的是：

```
1 s = "False"
2 if s:
3     print("Hello")
```

A Hello

B False

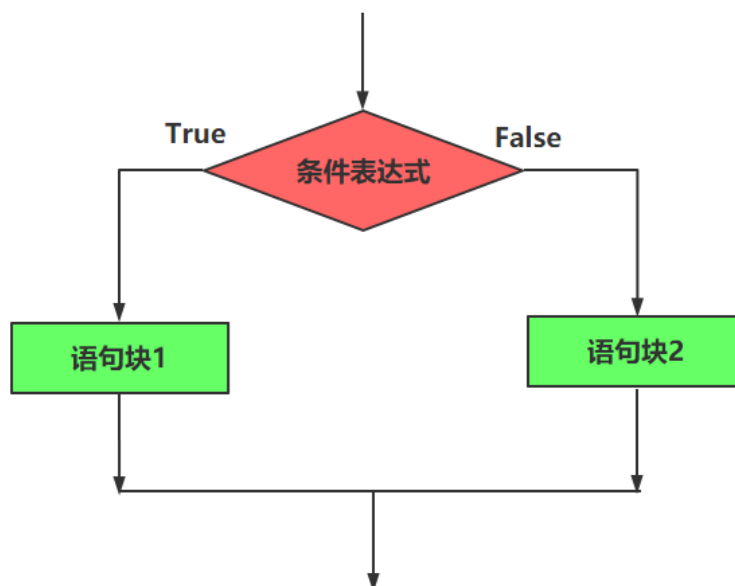
C 无打印输出

D "Hello"

答案

1=>A

双分支选择结构



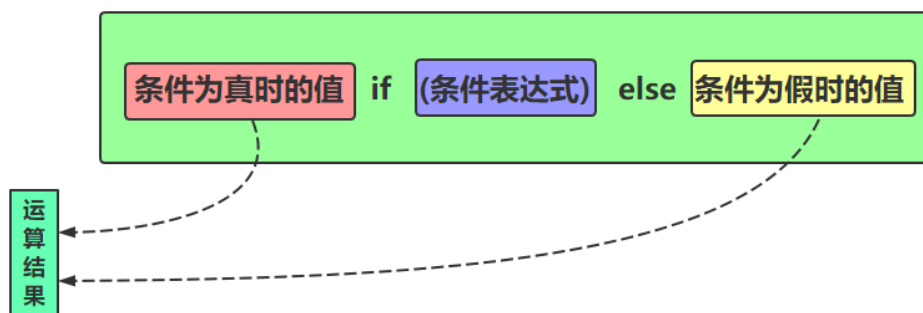
双分支结构的语法格式如下：

```
1 if 条件表达式:
2     语句1/语句块1
3 else:
4     语句2/语句块2
```

【操作】输入一个数字，小于10，则打印该数字；大于10，则打印“数字太大”

```
1 num = input("输入一个数字：")
2 if int(num)<10:
3     print(num)
4 else:
5     print("数字太大")
```

三元条件运算符



Python的三元运算符，用来在某些简单双分支赋值情况。三元条件运算符语法格式如下：

```
1 条件为真时的值  if  (条件表达式)  else  条件为假时的值
```

上一个案例代码，可以用三元条件运算符实现：

```
1 num = input("请输入一个数字")
2 print( num if int(num)<10 else "数字太大")
```

可以看到，这种写法更加简洁，易读。

实时效果反馈

1. 如下代码中if-else结果，可以用哪个代码替换：

```
1 a = input("输入一个数字： ")
2 a = int(a)
3 if a<10:
4     a = 0
5 else:
6     a = "数字太大"
```

A

a = 0 if a<10: "数字太大"

B

a = if a<10 else "数字太大"

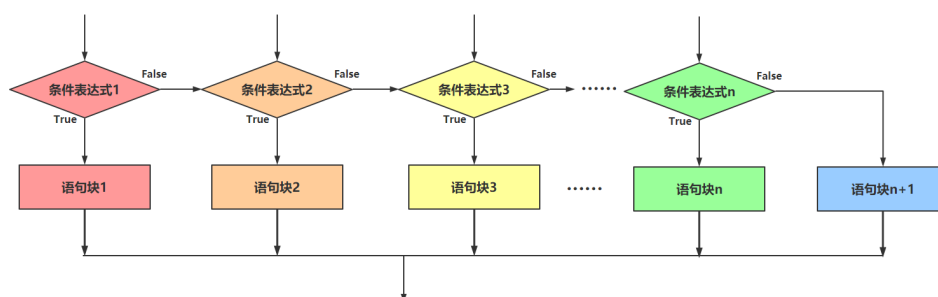
C `a = 0 if a < 10 else "数字太大"`

D `a = a if a < 10 else "数字太大"`

答案

1=>C

多分支选择结构



多分支选择结构的语法格式如下：

```

1  if 条件表达式1 :
2      语句1/语句块1
3  elif 条件表达式2:
4      语句2/语句块2
5  ...
6  elif 条件表达式n :
7      语句n/语句块n
8  [else:
9      语句n+1/语句块n+1
10 ]
  
```

注：计算机行业，描述语法格式时，使用中括号 `[]` 通常表示可选，非必选。

⚠ 多分支结构，几个分支之间是有逻辑关系的，不能随意颠倒顺序

【操作】输入一个学生的成绩，将其转化成简单描述：不及格(小于60)、及格(60-79)、良好(80-89)、优秀(90-100)

方法1（使用完整的条件表达）

```
1 score = int(input("请输入分数"))
2 grade = ''
3 if(score<60):
4     grade = "不及格"
5 if(60<=score<80):
6     grade = "及格"
7 if(80<=score<90):
8     grade = "良好"
9 if(90<=score<=100):
10    grade = "优秀"
11
12 print("分数是{0}, 等级是
    {1}".format(score, grade))
```

每个分支都使用了独立的、完整的判断，顺序可以随意挪动，而不影响程序运行。

方法2（利用多分支结构）

```
1 score = int(input("请输入分数"))
2 grade = ''
3 if score<60 :
4     grade = "不及格"
5 elif score<80 :
6     grade = "及格"
7 elif score<90 :
8     grade = "良好"
9 elif score<=100:
10    grade = "优秀"
11
12 print("分数是{0},等级是
    {1}".format(score,grade))
```

⚠ 多分支结构，几个分支之间是有逻辑关系的，不能随意颠倒顺序

【操作】已知点的坐标(x,y)，判断其所在的象限

```
1 x = int(input("请输入x坐标"))
2 y = int(input("请输入y坐标"))
3
4 if(x==0 and y==0):print("原点")
5 elif(x==0):print("y轴")
6 elif(y==0):print("x轴")
7 elif(x>0 and y>0):print("第一象限")
8 elif(x<0 and y>0):print("第二象限")
9 elif(x<0 and y<0):print("第三象限")
10 else:
11     print("第四象限")
```

实时效果反馈

1. 如下代码，若输入的分数是75，打印输出的是：

```
1 score = int(input("请输入分数"))
2 grade = ''
3 if score<60 :
4     grade = "不及格"
5 elif score<80 :
6     grade = "及格"
7 elif score<90 :
8     grade = "良好"
9 elif score<=100:
10    grade = "优秀"
11
12 print("分数是{0},等级是
    {1}".format(score,grade))
```

- ☐ A 不及格
- ☐ B 及格
- ☐ C 分数是75，等级是及格
- ☐ D 分数是75，等级是良好

答案

1=>C

选择结构嵌套

选择结构可以嵌套，使用时一定要注意控制好不同级别代码块的缩进量，因为缩进量决定了代码的从属关系。

【操作】输入一个分数。分数在0-100之间。90以上是A,80以上是B, 70以上是C, 60以上是D。60以下是E

```
1 score = int(input("请输入一个在0-100之间的数字: "))
2 grade = ""
3 if score>100 or score<0:
4     score = int(input("输入错误! 请重新输入一个在0-100之间的数字: "))
5 else:
6     if score>=90:
7         grade = "A"
8     elif score>=80:
9         grade = 'B'
10    elif score>=70:
11        grade = 'C'
12    elif score>=60:
13        grade = 'D'
14    else:
15        grade = 'E'
16
17 print("分数为{0}, 等级为{1}".format(score, grade))
```

或者也可以用下面代码更少的方法。不过，需要大家思考为什么这么写？

```
1 score = int(input("请输入一个在0-100之间的数字: "))
2 degree = "ABCDE"
3 num = 0
4 if score>100 or score<0:
5     score = int(input("输入错误! 请重新输入一个在0-100之间的数字: "))
6 else:
7     num = score//10
8     if num<6:num=5
9
10 print("分数是{0},等级是{1}".format(score,degree[9-num]))
```

循环结构

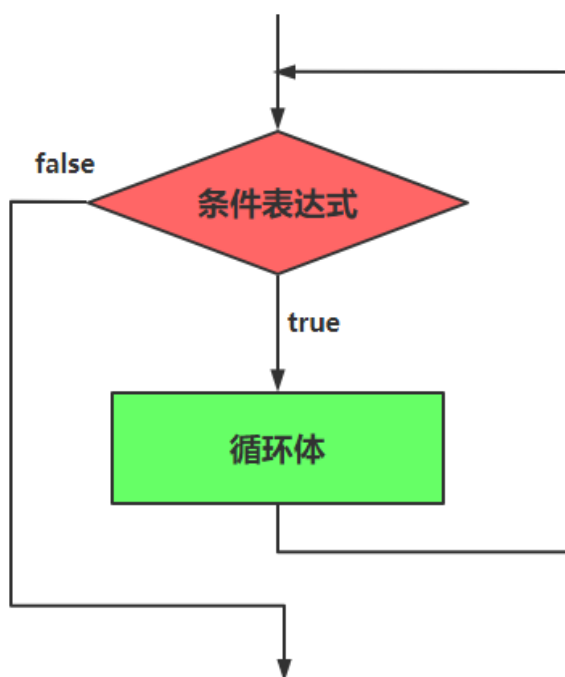
每天一句吃了么"
直到你爱上我为止



爱情大循环



循环结构用来重复执行一条或多条语句。表达这样的逻辑：如果符合条件，则反复执行循环体里的语句。在每次执行完后都会判断一次条件是否为True，如果为True则重复执行循环体里的语句。图示如下：



循环体里面的语句至少应该包含改变条件表达式的语句，以使循环趋于结束；否则，就会变成一个死循环。

while循环

while循环的语法格式如下：

```
1 while 条件表达式:
2     循环体语句
```

我们通过一些简单的练习，来慢慢熟悉while循环。

【操作】利用while循环打印从0-10的数字

```
1 num = 0
2 while num<=10:
3     print(num)
4     num += 1
```

【操作】利用while循环，计算1-100之间数字的累加和；计算1-100之间偶数的累加和，计算1-100之间奇数的累加和

```
1 num = 0
2 sum_all = 0           #1-100所有数的累加和
3 while num<=100:
4     sum_all += num
5     num += 1          #迭代，改变条件表达式，使循环
                        趋于结束
6
7 print("1-100所有数的累加和", sum_all)
```

实时效果反馈

1. 如下代码，计算1+2+...+100，空白处需要填入的代码是：

```

1 num = 0
2 sum_all = 0           #1-100所有数的累加和
3 while num<=100:
4     _____
5     num += 1          #迭代，改变条件表达式，使循环
                        趋于结束
6 print("1-100所有数的累加和",sum_all)

```

A `sum_all = num`

B `num+= sum_all`

C `sum_all = num+1`

D `sum_all += num`

答案

1=>D

for循环和可迭代对象遍历

for循环通常用于可迭代对象的遍历。for循环的语法格式如下：

```

1 for 变量 in 可迭代对象:
2     循环体语句

```

【操作】遍历一个元组或列表

```

1 for x in (20,30,40):
2     print(x*3)

```


可迭代对象

Python包含以下几种可迭代对象：

- 1 序列。包含：字符串、列表、元组、字典、集合
- 2 迭代器对象 (iterator)
- 3 生成器函数 (generator)
- 4 文件对象

我们已经在前面学习了序列、字典等知识，迭代器对象和生成器函数将在后面进行详解。接下来，我们通过循环来遍历这几种类型的数据：

【操作】遍历字符串中的字符

```
1 for x in "sxt001":  
2     print(x)
```

【操作】遍历字典

```
1 d = {'name': 'gaoqi', 'age': 18, 'address': '西三  
   旗001号楼'}  
2 for x in d: #遍历字典所有的key  
3     print(x)  
4  
5 for x in d.keys(): #遍历字典所有的key  
6     print(x)  
7  
8 for x in d.values(): #遍历字典所有的value  
9     print(x)  
10  
11 for x in d.items(): #遍历字典所有的"键值对"  
12     print(x)
```

range对象

`range`对象是一个迭代器对象，用来产生指定范围的数字序列。格式为：

```
1 range(start, end [,step])
```

生成的数值序列从 `start` 开始到 `end` 结束（`!` 不包含 `end`）。若没有填写 `start`，则默认从0开始。`step` 是可选的步长，默认为1。如下是几种典型示例：

```
for i in range(10)           产生序列：0 1 2 3 4 5 6 7 8 9
```

```
for i in range(3,10)        产生序列：3 4 5 6 7 8 9
```

```
for i in range(3,10,2)      产生序列：3 5 7 9
```

【操作】利用for循环，计算1-100之间数字的累加和；计算1-100之间偶数的累加和，计算1-100之间奇数的累加和。

```
1 sum_all = 0           #1-100所有数的累加和
2 sum_even = 0          #1-100偶数的累加和
3 sum_odd = 0           #1-100奇数的累加和
4 for num in range(101):
5     sum_all += num
6     if num%2==0:sum_even += num
7     else:sum_odd += num
8
9 print("1-100累加总和{0}, 奇数和{1}, 偶数和{2}".format(sum_all,sum_odd,sum_even))
```

实时效果反馈

1. 如下代码，空白处的代码是：

```
1 sum_all = 0                #1-100所有数的累加和
2 _____:
3     sum_all += num
4 print("1-100累加总和{0}".format(sum_all))
```

A `for i in range(101)`

B `for i in range(100)`

C `for num in range(101)`

D `for num in range(100)`

答案

1=>C

嵌套循环和综合练习



一个循环体内可以嵌入另一个循环，一般称为“嵌套循环”，或者“多重循环”。

【操作】打印如下图案

```

0  0  0  0  0
1  1  1  1  1
2  2  2  2  2
3  3  3  3  3
4  4  4  4  4

```

```

1 for x in range(5):
2     for y in range(5):
3         print(x,end="\t")
4     print()    #仅用于换行

```

【操作】利用嵌套循环打印九九乘法表

```

1 for m in range(1,10):
2     for n in range(1,m+1):
3         print("{0}*{1}={2}".format(m,n,
4             (m*n)),end="\t")
5     print()

```

九九乘法表:

```

1*1=1
2*1=2  2*2=4
3*1=3  3*2=6  3*3=9
4*1=4  4*2=8  4*3=12  4*4=16
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
7*1=7  7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81

```

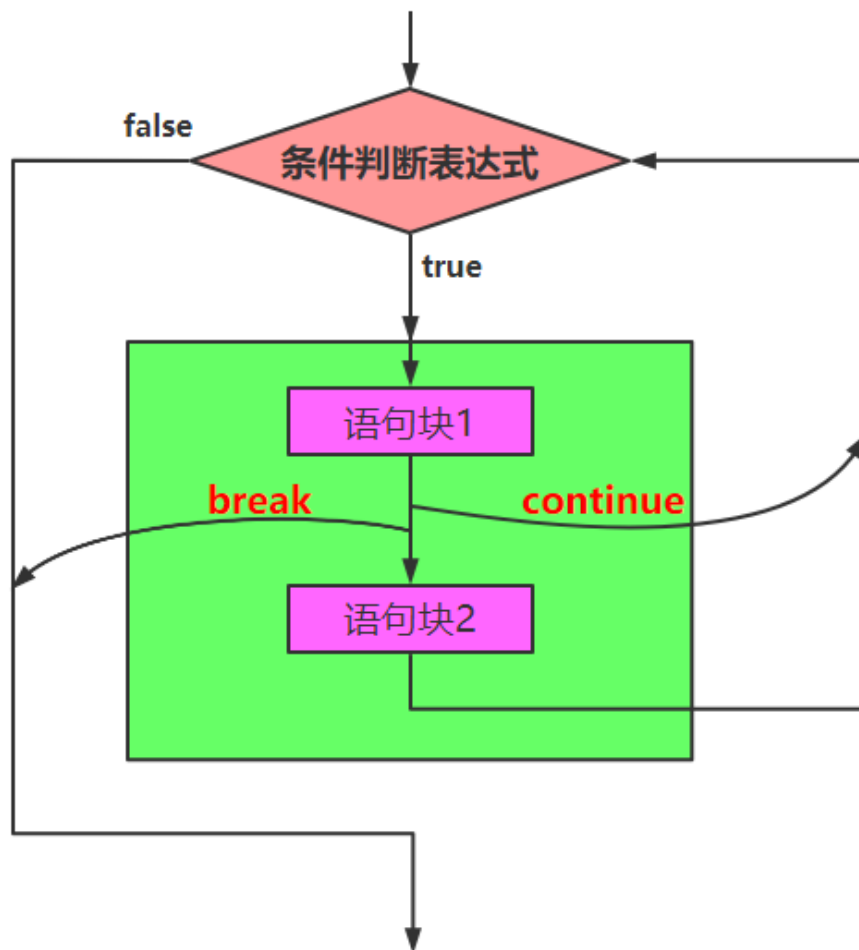
【操作】用列表和字典存储下表信息，并打印出表中工资高于15000的数据

姓名	年龄	薪资	城市
高小一	18	30000	北京
高小二	19	20000	上海
高小五	20	10000	深圳

```
1 r1= dict(name="高小  
一",age=18,salary=30000,city="北京")  
2 r2= dict(name="高小  
二",age=19,salary=20000,city="上海")  
3 r3= dict(name="高小  
三",age=20,salary=10000,city="深圳")  
4 tb = [r1,r2,r3]  
5  
6 for x in tb:  
7     if x.get("salary")>15000:  
8         print(x)
```

break语句

break语句可用于while和for循环，用来结束整个循环。当有嵌套循环时，break语句只能跳出最近一层的循环。



【操作】使用break语句结束循环

```

1 while True:
2     a = input("请输入一个字符（输入Q或q结束）")
3     if a.upper()=='Q':
4         print("循环结束，退出")
5         break
6     else:
7         print(a)

```

实时效果反馈

1. 循环结构中，break的作用是：

A 用来结束整个循环

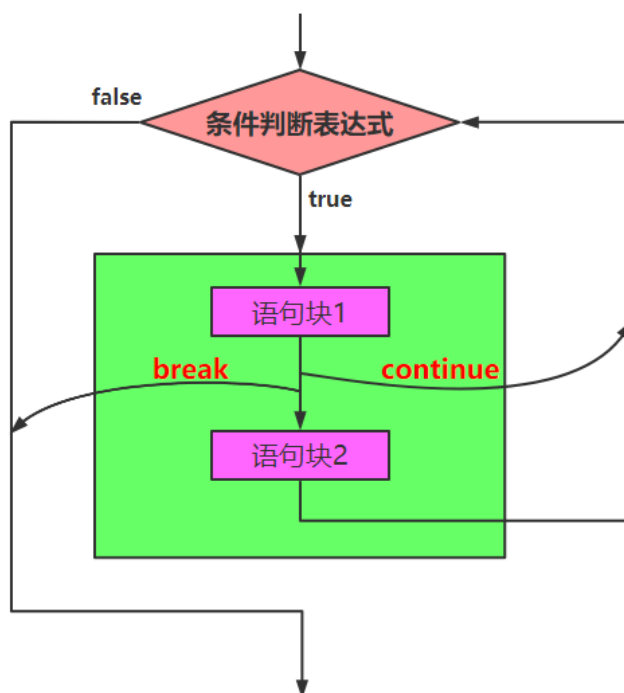
- B** 用来结束本次循环，继续下一次循环
- C** 用来暂停整个循环
- D** 休息一下

答案

1=>A

continue语句

continue语句用于结束本次循环，继续下一次。多个循环嵌套时，continue也是应用于最近的一层循环。



【操作】要求输入员工的薪资，若薪资小于0则重新输入。最后打印出录入员工的数量和薪资明细，以及平均薪资

```
1 empNum = 0
2 salarySum= 0
3 salarys = []
4 while True:
5     s = input("请输入员工的薪资（按Q或q结束）")
6
7     if s.upper()=='Q':
8         print("录入结束")
9         break
10    if float(s)<0:
11        print("无效！继续录入！")
12        continue
13    print("录入成功！")
14    empNum +=1
15    salarys.append(float(s))
16    salarySum += float(s)
17
18 print("员工数{0}".format(empNum))
19 print("录入薪资：",salarys)
20 print("总发薪资：",salarySum)
21 print("平均薪资{0}".format(salarySum/empNum))
```

执行结果：

请输入员工的薪资（按Q或q结束） 2000

请输入员工的薪资（按Q或q结束） 3000

请输入员工的薪资（按Q或q结束） 4000

请输入员工的薪资（按Q或q结束） 5000

请输入员工的薪资（按Q或q结束） Q

录入完成，退出

员工数4

录入薪资: [2000.0, 3000.0, 4000.0, 5000.0]

平均薪资3500.0

实时效果反馈

1. 循环结构中，continue的作用是：

- A** 用来结束整个循环
- B** 用来结束本次循环，继续下一次循环
- C** 用来暂停整个循环
- D** 休息一下

答案

1=>B

else语句

while、for循环可以附带一个else语句（可选）。如果for、while语句没有被break语句结束，则会执行else子句，否则不执行。语法格式如下：

```
1 while 条件表达式:
2     循环体
3 else:
4     语句块
5
6 或者:
7 for 变量 in 可迭代对象:
8     循环体
9 else:
10    语句块
```

【操作】员工一共4人。录入这4位员工的薪资。全部录入后，打印提示“您已经全部录入4名员工的薪资”。最后，打印输出录入的薪资和平均薪资

```
1 salarySum= 0
2 salarys = []
3 for i in range(4):
4     s = input("请输入一共4名员工的薪资（按Q或q中途
5         结束）")
6     if s.upper()=='Q':
7         print("录入完成，退出")
8         break
9     if float(s)<0:
10        continue
11
12    salarys.append(float(s))
13    salarySum += float(s)
14
15 else:
16    print("您已经全部录入4名员工的薪资")
```

```
17 |
18 | print("录入薪资:", salarys)
19 | print("平均薪资{0}".format(salarySum/4))
```

实时效果反馈

1. 循环结构中，else的作用是：

- A** 碰到break语句，则会执行else子句
- B** 无论有无continue，一定会执行else子句
- C** 无论有无break，一定会执行else子句
- D** 如果for、while语句没有被break语句结束，则会执行else子句，否则不执行

答案

1=>D

循环代码优化

虽然计算机越来越快，空间也越来越大，我们仍然要在性能问题上“斤斤计较”。编写循环时，遵守下面三个原则可以大大提高运行效率，避免不必要的低效计算：

- ① 尽量减少循环内部不必要的计算
- ② 嵌套循环中，尽量减少内层循环的计算，尽可能向外提
- ③ 局部变量查询较快，尽量使用局部变量

```
1 | #循环代码优化测试
2 | import time
3 |
```

```

4 start = time.time()
5 for i in range(1000):
6     result = []
7     for m in range(10000):
8         c = i * 1000 #提到外部循环
9         #result = result + [m * 100] #不使用拼
            接,会产生大量新对象
10        result.append(c+m*100)
11 end = time.time()
12 print("耗时: {0}".format((end-start)))
13
14 print("简单循环优化后...")
15 start2 = time.time()
16 for i in range(1000):
17     result = []
18     c = i*1000
19     for m in range(10000):
20         result.append(c+m*100)
21
22 end2 = time.time()
23 print("耗时: {0}".format((end2-start2)))

```

其他优化手段

- ① 连接多个字符串, 使用join()而不使用+
- ② 列表进行元素插入和删除, 尽量在列表尾部操作

实时效果反馈

1. 编写循环时, 代码优化规则说法错误的是:

- A** 嵌套循环中, 尽量减少内层循环的计算, 尽可能向外提

- B** 局部变量查询较快，尽量使用局部变量
- C** 连接多个列表，使用+而不使用join()
- D** 列表进行元素插入和删除，尽量在列表尾部操作

答案

1=>C

使用zip()并行迭代多个序列

我们可以通过zip()函数对多个序列进行并行迭代，zip()函数在最短序列“用完”时就会停止。

【操作】测试zip()并行迭代

```
1 names = ("高淇", "高老二", "高老三", "高老四")
2 ages = (18, 16, 20, 25)
3 jobs = ("老师", "程序员", "公务员")
4
5 for name, age, job in zip(names, ages, jobs):
6     print("{0}--{1}--{2}".format(name, age, job))
7
8 #不适用zip，也可以并行迭代多个序列
9 for i in
10    range(min(len(names), len(ages), len(jobs))):
11        print("{0}--{1}--{2}".format(names[i], ages[i], jobs[i]))
```

执行结果：

高淇--18--老师

高老二--16--程序员

高老三--20--公务员

推导式创建序列

推导式是从一个或者多个迭代器快速创建序列的一种方法。它可以将循环和条件判断结合，从而避免冗长的代码。



♡ 推导式是典型的Python风格，会使用它，代表你已经超过Python初学者的水平。

列表推导式

列表推导式生成列表对象，语法如下：

```

1 [表达式 for item in 可迭代对象 ]
2 或者: {表达式 for item in 可迭代对象 if 条件
   判断}

```

```

1 [x for x in range(1,5)] #[1, 2, 3, 4]
2 [x*2 for x in range(1,5)] #[2, 4, 6, 8]
3 [x*2 for x in range(1,20) if x%5==0 ] #[10,
   20, 30]
4 [a for a in "ABCDEFGH"] #['A', 'B', 'C',
   'D', 'E', 'F', 'G']
5 #可以使用两个循环,使用zip并行迭代
6 cells = [(row,col) for row,col in
   zip(range(1,10),range(101,110))]
7
8 print(cells)

```

```

1 a = [x for x in range(1,10) if x%2==0]
2 print(a)
3
4 #不适用推导式, 生成列表。二者对比。体现推导式的优势
5 b = []
6 for x in range(1,10):
7     if x%2==0:
8         b.append(x)
9 print(b)

```

字典推导式

字典的推导式生成字典对象, 格式如下:

```
1 {key_expression: value_expression for 表达式
   in 可迭代对象}
```

类似于列表推导式，字典推导也可以增加if条件判断、多个for循环。

```
1 values = ["北京", "上海", "深圳", "广州"]
2 cities = {id*100:city for id,city in
            zip(range(1,5),values)}
3 print(cities)
```

生成字典对象：

```
1 {100: '北京', 200: '上海', 300: '深圳', 400:
   '广州'}
```

统计文本中字符出现的次数：

```
1 my_text = ' i love you, i love sxt, i love
   gaoqi '
2 char_count = {c:my_text.count(c) for c in
                my_text}
3 print(char_count)
```

结果输出：

```
1 {' ': 9, 'i': 4, 'l': 3, 'o': 5, 'v': 3, 'e':
   3, 'y': 1, 'u': 1, ',': 2, 's': 1, 'x': 1,
   't': 1, 'g': 1, 'a': 1, 'q': 1}
```


集合推导式

集合推导式生成集合，和列表推导式的语法格式类似：

```
1 {表达式 for item in 可迭代对象 }
2 或者：{表达式 for item in 可迭代对象 if 条件判断}
```

```
1 >>> {x for x in range(1,100) if x%9==0}
2 {99, 36, 72, 9, 45, 81, 18, 54, 90, 27, 63}
```

生成器推导式（不直接生成元组）

很多同学可能会问：“都有推导式，元组有没有？”，能不能用小括号呢？

```
1 >>> (x for x in range(1,100) if x%9==0)
2 <generator object <genexpr> at
  0x0000000002BD3048>
```

我们发现提示的是“一个生成器对象”。显然，元组是没有推导式的。

一个生成器只能运行一次。第一次迭代可以得到数据，第二次迭代发现数据已经没有了。

```
1 gnt = (x for x in range(1,100) if x%9==0)
2 for x in gnt:
3     print(x,end=' ')
4 for x in gnt:
5     print(x,end=' ')
```

1. 如下列表推导式的代码，输出的是：

```
1 [x*2 for x in range(1,20) if x%5==0 ]
```

A [5,10,15,20,25]

B [5,10,15, 20, 25,30]

C [1, 2, 3]

D [10, 20, 30]

2. 如下是字典推导式代码，输出的是：

```
1 values = ["北京","上海","深圳","广州"]
2 cities = {id*100:city for id,city in
zip(range(1,5),values)}
3 print(cities)
```

A {'北京':1, '上海':2, '深圳':3, '广州':4}

B {'北京':100, '上海':200, '深圳':300, '广州':400}

C {100: '北京', 200: '上海', 300: '深圳', 400: '广州'}

D {1: '北京', 2: '上海', 3: '深圳', 4: '广州'}

答案

1=>D 2=>C

综合练习



① 【操作】 绘制多个同心圆

```
1 import turtle
2
3 p = turtle.Pen() #画笔对象
4
5 radius = [x*10 for x in range(1,11)]
6 #10,20,30,40...
7 my_colors =
8 ("red", "green", "yellow", "black")
9 p.width(4)
10
11 for r,i in zip(radius,range(len(radius))):
12     p.penup()
13     p.goto(0,-r)
14     p.pendown()
15     p.color(my_colors[i%len(my_colors)])
16     p.circle(r)
17
18 turtle.done() #程序执行完毕，窗口在
```

② 【操作】 绘制18*18棋盘

```
1 #画棋盘
2 import turtle
3
4 width = 30
5 num = 18
6
7 x1 = [(-400,400),(-400+width*num,400)]
8 y1 = [(-400,400),(-400,400-width*num)]
9 t = turtle.Pen()
10 t.speed(10)
11 # t.goto(x1[0][0],x1[0][1])
12 # t.goto(x1[1][0],x1[1][1])
13
14 for i in range(num+1):
15     t.penup()
16     t.goto(x1[0][0],x1[0][1]-30*i)
17     t.pendown()
18     t.goto(x1[1][0],x1[1][1]-30*i)
19
20 for i in range(num+1):
```

```

21     t.penup()
22     t.goto(y1[0][0]+30*i,y1[0][1])
23     t.pendown()
24     t.goto(y1[1][0]+30*i,y1[1][1])
25
26 t.hideturtle()    #隐藏画笔
27 turtle.done()    #保证运行窗口不被自动关闭

```

实操作业

- ① 输入一个学生的成绩，将其转化成简单描述：不及格(小于60)、及格(60-79)、良好(80-89)、优秀(90-100)
- ② 已知点的坐标(x,y)，判断其所在的象限
- ③ 输入一个分数。分数在0-100之间。90以上是A,80以上是B，70以上是C，60以上是D。60以下是E
- ④ 利用while循环，计算1-100之间数字的累加和；计算1-100之间偶数的累加和，计算1-100之间奇数的累加和
- ⑤ 利用for循环，计算1-100之间数字的累加和；计算1-100之间偶数的累加和，计算1-100之间奇数的累加和
- ⑥ 打印如下图案：

```

0    0    0    0    0
1    1    1    1    1
2    2    2    2    2
3    3    3    3    3
4    4    4    4    4

```

- ⑦ 利用嵌套循环打印九九乘法表

- 8 用列表和字典存储下表信息，并打印出表中工资高于15000的数据

高小一	18	30000	北京
高小二	19	20000	上海
高小五	20	10000	深圳

- 9 要求输入员工的薪资，若薪资小于0则重新输入。最后打印出录入员工的数量和薪资明细，以及平均薪资
- 10 员工一共4人。录入这4位员工的薪资。全部录入后，打印提示“您已经全部录入4名员工的薪资”。最后，打印输出录入的薪资和平均薪资
- 11 使用海龟绘图，绘制同心圆：



- 12 使用海龟绘图，绘制18*18的棋盘：

